

Performance Evaluation of a Dynamic RESTful API Using FastAPI, Docker and Nginx

Ebenhezer Mabotha

Department of Electrical and Electronic Technology

University of Johannesburg
Johannesburg, South Africa
emabotha@gmail.com

Nkateko E Mabunda

Department of Electrical and Electronic Technology

University of Johannesburg
Johannesburg, South Africa
nkatekom@uj.ac.za

Ahmed Ali

Department of Electrical and Electronic Technology

University of Johannesburg
Johannesburg, South Africa
aali@uj.ac.za

Abstract— This paper provides a performance evaluation of a dynamic RESTful API architecture suitable for IoT deployments. The framework, developed with Python's FastAPI, PostgreSQL, and Nginx and containerized with Docker, was tested for flexibility, reliability, and efficiency under various scenarios. The evaluation focused on key operational indicators such as response time, throughput, load handling, and security. Functional testing confirmed the functionality of essential API endpoints used for dynamic CRUD operations, providing consistent response accuracy. Load testing using Apache JMeter revealed that the system maintained an average response time of 95ms under moderate loads and could handle many concurrent requests with negligible performance deterioration, demonstrating its ability. The throughput remained consistent even at high traffic volumes, demonstrating its suitability for dynamic IoT applications. Security testing, aided by Nginx's rate-limiting feature, demonstrated the framework's resistance to popular threats such as SQL injection and DDoS. The results show that the dynamic API framework is ideal for expandable IoT installations, providing high availability, low latency, and responsiveness in real-time applications. These results demonstrate its capacity to handle demanding IoT scenarios while maintaining performance dependability and security. This paper provides the testing methods and performance results contributing to the framework's fit for real-world IoT use cases.

Keywords—RESTful, API, IoT, Docker, Deployment, Interface, JMeter, FastAPI

I. INTRODUCTION

The Internet of Things (IoT) expansion has resulted in large amounts of data generated by networked devices, demanding RESTful APIs that can manage such data effectively. RESTful APIs, noted for their simplicity and statelessness, are widely used to connect IoT devices to servers [1]. Due to the dynamic nature of IoT environments, traditional API frameworks frequently experience flexibility and performance issues, particularly in high-traffic situations. These environments are distinguished by variable data loads and the need for real-time processing [2].

This article tackles these issues by evaluating the performance of a dynamic RESTful API built on the FastAPI framework and optimized for IoT systems. FastAPI's asynchronous characteristics enable it to handle several requests simultaneously while maintaining speed and reliability. It is ideal for data-intensive applications such as smart cities, industrial automation, and healthcare monitoring [3]. The API's architecture is intended to adapt to changing demand levels while maintaining peak

performance. Since it follows RESTful principles, it efficiently performs CRUD (Create, Read, Update, and Delete) activities on various endpoints. Reliable authentication procedures also protect data and ensure that only authorized users can access or modify it [4].

The primary goal of this study is to conduct a thorough performance investigation of the API in a controlled environment. Key parameters such as throughput, response time, status codes and load testing are tested in various traffic scenarios to imitate real-world situations and identify potential bottlenecks. The findings provide insight into the API's performance, offering recommendations for large-scale IoT deployments.

The findings have consequences for those aiming to design flexible, high-performance APIs in situations where reliability is essential. The results show that FastAPI provides a suitable foundation for creating dynamic APIs that can adapt to the changing demands of IoT networks [5].

II. RELATED WORKS

The development of reliable and flexible web services relies heavily on careful API testing and evaluation. Research has demonstrated that assessing API performance, throughput, response time, and dependability is essential for guaranteeing scalability and reliability under fluctuating loads. Continuous integration and deployment methodologies, like those mentioned by [6], place a strong emphasis on automated testing to preserve system stability. Newman [7] addresses the function of microservices in facilitating flexible APIs, and some have investigated cloud-based environments that improve API responsiveness and throughput.

API testing is essential for ensuring that an API meets its functional, performance, and reliability criteria. Unit testing, integration testing, and end-to-end testing are common techniques for validating API functionality. However, performance testing, which includes monitoring the API's response time, throughput, and resource consumption under various loads, is especially important in dynamic and high-traffic scenarios [8]. API performance testing has made broad use of tools such as JMeter, Postman, and LoadRunner. The reliability of JMeter, an open-source application, in simulating severe loads on servers, networks, or objects to evaluate their strength and examine overall performance under various load types is well known as detailed by [9][10][11]. With its support for many testing methodologies, including load, stress, and endurance testing, it offers comprehensive insights into

resource usage, throughput, and API response times [12]. However, because it provides complete automated testing solutions, including advanced scripting and monitoring features, LoadRunner is frequently utilized in large-scale systems [13]. From e-commerce to Internet of Things applications, these tools have proven essential in assessing the performance of APIs.

An important statistic in assessing the performance of an API is throughput, which is a measurement of the quantity of transactions a system can process in a specified amount of time. Research like [14] have demonstrated that managing server resources, such as CPU and memory, carefully and effectively handling multiple concurrent requests is necessary for throughput optimization. Another key measure directly related to user satisfaction and system usability is response time, or how long it takes an API to react to a request. A short response time API is generally preferred since it guarantees a faster client-server exchange, which is crucial for real-time applications. Smith et al. [15] stress the relevance of adaptive algorithms in improving API response times, particularly in dynamic contexts with unpredictable traffic. Asynchronous processing and load-balancing techniques have significantly enhanced response times, ensuring that APIs remain responsive even when heavily used.

Another important part of API testing is reliability, which is frequently defined by the API's capacity to manage faults while maintaining consistent performance. Fault injection testing, which introduces controlled faults into the system to verify its robustness, is a typical technique for assessing dependability. Scalability, or an API's capacity to accommodate increased loads by adding resources, is strongly related to reliability. APIs that are both dependable and scalable are more likely to deliver consistent performance under changing operational conditions. API stability and scalability are especially important in IoT applications that rely heavily on dynamic data interchange. The work done by [16] found that microservices architectures improve the scalability and stability of APIs in IoT systems. Microservices architectures enable more granular scaling and enhanced fault isolation by dividing the API into smaller, independent services, which improves system performance as a whole.

Besides from classic tools like Jmeter, new approaches and tools are being investigated for API testing. Containerization solutions, such as Docker, have been popular in recent years. Docker enables the establishment of isolated environments in which APIs can be tested under controlled circumstances [17]. This strategy not only allows for more consistent testing results, but it also simplifies the process of recreating test settings. AI and machine learning advances are being integrated into API testing tools to detect possible bottlenecks and automatically enhance performance. These complex instruments may automatically alter the testing parameters based on real-time input, offering more accurate and useful insights regarding API performance [18].

In summary, evaluating API performance is a comprehensive procedure that includes assessing a variety of parameters such as functional, throughput, response time, and reliability. The use of advanced tools and processes is essential for ensuring that APIs match the expectations of modern applications, especially in dynamic and high-traffic

contexts such as IoT systems. The ongoing growth of API testing tools, fueled by advances in AI, containerization, and microservices, promises to improve the accuracy and efficiency of API performance measurement.

III. METHODOLOGY

This section discusses the Dynamic API framework design and methods that were used in evaluating its performance.

1. Architectural design

The dynamic RESTful API framework's architectural design is based on microservices and containerization principles, which promote modularity, scalability, and maintainability. As described in [19], the microservices design divides complicated systems into smaller, loosely linked services, improving development and deployment efficiency. Docker containers, as supported by [20], allow lightweight and portable encapsulation, resulting in consistent and reproducible deployments across several settings.

Figure 1 illustrates the framework's architecture, with the Nginx container serving as the primary entry point, handling routing and rate limiting. As a reverse proxy, Nginx effectively directs incoming requests and enforces rate limitations to keep the system running smoothly, as demonstrated by [21]. The interface container, which hosts the FastAPI application and PostgreSQL database, is the API's principal point of engagement. As recommended by [22] and [23], this configuration ensures effective data management and operational reliability. The API client container, represents all the applications that interacts with the API specifically Postman and JMeter.

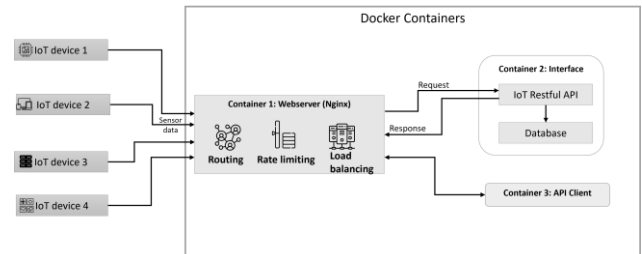


Figure 1: Architectural design

2. Dynamic Restful API

The dynamic RESTful API framework, developed with Python and FastAPI, was designed to handle various CRUD activities via a set of well-defined endpoints. Based on the REST architectural approach described by [2] and [24], the API supports authentication endpoints for user administration alongside six dynamic related endpoints for CRUD operations. The endpoints are detailed below:

2.1. Adding an interface

The **POST** method to the **/dynamic** route facilitates the creation of new dynamic interfaces. The request payload, includes the endpoint URL, description, and associated database table name. The application validates the payload and creates a corresponding database table, ensuring proper structure and validation.

2.2. Retrieving an Interface

To access information about all dynamic interfaces associated with a user, the API uses the **GET** method via the

/dynamic endpoint. This request enables users to obtain a complete list of their interfaces. The response consists of information about the interface URL, ID, description, and a list of fields associated with each interface. This retrieval method ensures that users can access and examine all dynamic interfaces they have created or are linked to.

2.3. Updating an Interface

To update an existing interface, use the **PUT** method at the **/dynamic** endpoint. This method enables users to change or remove fields in an interface based on the new data in the request payload. The API modifies and validates the interface's structure to ensure it fits the new requirements. This includes ensuring that the changes comply with the updated data requirements and reflect them in the database. The changed structure is then saved and reused for other interactions with the interface.

2.4. Deleting an Endpoint

To remove an existing interface, use the **DELETE** method at the **/dynamic** endpoint. The process starts with authenticating the user to ensure they have the relevant permissions. Following authentication, the API confirms the presence of the interface that will be erased. If the interface is identified, the API removes the related database table and the interface entry from the system. This ensures that the interface and associated data are removed from the system, as per the user's request.

2.5. Adding Interface Data

To add data/history to a specific endpoint, use the **POST** method with the URL **/dynamic/interface/**. The request payload includes the data fields specified when the interface was created. The API checks the user's information before processing the data entry. Upon successful validation, the data is added to the database table associated with the interface URL, ensuring that it is properly saved and linked to the chosen endpoint.

2.6. Getting Interface Data

To retrieve data from a specific endpoint, use the **GET** method on the **/dynamic/interface/** route. This call allows users to retrieve data linked with a specific interface. The API handles the request by confirming the user's credentials before returning the data stored for that endpoint. This response provides all essential data fields and values, giving a complete picture of the data associated with the requested interface.

3. Building of Docker containers

The dynamic RESTful API framework was deployed by containerizing its components with Docker to provide consistent and portable execution across several environments. Each container has a Dockerfile that describes the required environment, dependencies, and parameters.

3.1. Nginx Container

The Nginx container is responsible for managing incoming HTTP requests and routing them to the appropriate backend services. The Dockerfile for the Nginx container is illustrated in Figure 2.

```
# Use the official Nginx image as the base
image FROM nginx:latest

# Install Certbot and the Nginx plugin
RUN apt-get update && apt-get install -y
certbot python3-certbot-nginx

# Copy the custom Nginx configuration file
into the container
# Ensure you have a custom nginx.conf file in
the same directory as this Dockerfile
COPY ./config/default.conf
/etc/nginx/nginx.conf

# Expose port 80 and 443 for HTTP and HTTPS
EXPOSE 80 443
```

Figure 2: Nginx Docker file

The Nginx Docker file can be summarized as follows:

- The base image `nginx:latest` is used to ensure that the container uses the most recent stable version of Nginx.
- CertBot and the Nginx plugin are installed to support SSL/TLS certificates, which improves communication security.
- A custom Nginx configuration file, `default.conf`, is copied into the container to specify how requests will be processed.
- Ports 80 and 443 are open for HTTP and HTTPS communication to reach the container.

The custom Nginx configuration file, **default.conf**, includes the following settings:

- Rate Limiting: To prevent abuse, limit requests to 10 per second.
- Proxy Headers: Ensures that client data is sent to backend services.
- Location Blocks: Determines how different pathways are handled, such as supplying static content, managing documentation, and proxying requests to backend services.
- Error Handling: Customizes replies for typical errors like 404 and 50x.

3.2. Interface container

The Interface container includes both PostgreSQL and a FastAPI application. The Dockerfile for this container can be summarized as follows:

- Base Image and Environment Setup: Uses Ubuntu 22.04 and sets non-interactive mode for package installations to avoid prompts.
- Package Installation: Installs PostgreSQL, additional utilities, and cleans up to reduce the image size.
- PostgreSQL Configuration: Sets environment variables for PostgreSQL and creates the application directory.
- Python Application Setup: Installs Python, FastAPI, and other dependencies, and copies application files into the container.

- Port Exposure: Exposes ports 5432 and 8888 for PostgreSQL and FastAPI services.
- SQL Script Customization and Initialization: Updates the PostgreSQL password placeholder and initializes the database before starting the FastAPI application.

4. Deployment

The dynamic RESTful API framework was deployed on an Ubuntu 24.04 server, which was chosen for its reliability and compatibility with modern software tools. The server configuration was set up to meet the system requirements for a valid performance evaluation. The system had a dual-core processor, 2 GB of RAM, and 75 GB of SSD storage. This setup was chosen to evaluate whether the framework can withstand moderate to heavy traffic while correctly modeling real-world deployment conditions.

Docker Compose was used to orchestrate the container deployments, making it easier to manage the multi-container configuration. This strategy created an easily replicable setting, which is critical for performing reliable performance evaluations. The Docker Compose configuration for this setup is shown below:

```
version: '3'

networks:
  interface_network:

services:
  webservice:
    build: ./webservice
    ports:
      - "80:80"
      - "443:443"
    networks:
      - interface_network
  interface:
    build: ./interface
    ports:
      - "5432:5432"
      - "8888:8888"
    environment:
      - POSTGRES_PASSWORD=$POSTGRES_PASSWORD #
        Set your desired password here
    networks:
      - interface_network
```

Figure 3: Deployment docker-compose file

5. Performance evaluation

The dynamic RESTful API's performance evaluation methodology was based on an organized process that guarantees thorough testing in all areas, from security to functionality. The main techniques and procedures for assessing the dependability and performance of the API are described below.

5.1. Authentication and authorization

The authentication and permission testing for the Dynamic RESTful API was developed to ensure that only valid and authorized users may access and use the API securely. The testing process started with user registration. When a user registers, they provide a valid email address and a password via the registration API endpoint. When the user successfully registers, the system sends a validation email with a unique link to their email address. The link in the email must be clicked to verify the email address and activate the account. The testing procedure ensured that the

validation email was delivered correctly, and logs were reviewed to ensure that the email was sent successfully. Special scenarios, such as processing incorrect email addresses and managing requests for resending validation emails, were also tested to ensure system's reliability.

Once the user's email address has been verified, the user is authorized to log in. On login, the user submits their email address and password, and after successful authentication, the system generates a unique *api_key* for that user. This *api_key* acts as a token and must be supplied in the header of any subsequent API calls to authenticate the user and grant access to the API's endpoints. The security of this process was validated by ensuring that the *api_key* was produced appropriately upon login, securely maintained, and properly invalidated upon logout or session expiration. Unauthorized access attempts, such as those with invalid or expired *api_keys*, were also assessed to ensure that the API delivered suitable error answers and refused access to secure resources.

5.2. Status code validation

The status code verification process was carried out to determine the API endpoints' responses and reliability in handling various types of queries. This step is important for ensuring that the API returns the correct HTTP status code based on the kind of request and its results. The purpose of testing several endpoints with a variety of valid and invalid inputs was to ensure that each endpoint behaved as expected under different scenarios such as successful operations, unauthorized access, resource unavailability, and validation failures.

Each endpoint was subjected to several test cases that simulated real-world conditions. For example, POST requests were tested to ensure that data was created and sent, and GET requests were reviewed to retrieve data correctly. PUT and DELETE requests were used to evaluate the handling of updates and deletions. These tests validated the API's ability to handle errors and provide correct responses.

The testing process validated that the API followed traditional standards by returning accurate status codes, which are important to client-server communication. Correct status codes inform clients about the success or failure of requests, allowing them to implement effective error handling and improve the user experience. This method is also important for identifying possible difficulties with data validation, resource management, and security, as well as providing insights into areas for future optimization.

5.3. Function testing

To ensure that every API endpoint carries out its intended functions as intended, functional testing is the main emphasis of the assessment process's initial step. In this process, Postman was used to create test cases for each endpoint, including POST, GET, PUT, and DELETE. Every endpoint undergoes extensive testing to guarantee that it can carry out necessary tasks, including setting up, obtaining, updating, and removing dynamic interfaces.

During functional testing, response validation was important and therefore the Nginx rate limit was not considered in this test. The API responses were compared to the expected status codes and payloads, as provided in the API documentation. This phase verified that the API consistently returned the appropriate data structure and values. The API's error-handling capabilities were tested

using incorrect inputs and exceptions. The system's ability to return relevant error messages and gracefully manage exceptions was tested to verify reliability in real-world circumstances.

5.4. Load testing

A series of load tests were run to assess the performance of a Nginx server that was configured with a rate-limiting rule. The server's rate-limiting was configured as illustrated in Figure 4:

```
limit_req_zone $binary_remote_addr
zone=mylimit:10m rate=10r/s;
```

Figure 4: Nginx rate limit setting

This setting limited each IP address to a maximum of ten requests per second. The primary goal of the test was to determine how the server responded to traffic loads that exceeded the rate limit, as well as its capacity to maintain steady performance under sustained load.

To simulate changing traffic levels, JMeter was used to produce a growing number of requests while analyzing important performance metrics like response times, throughput, and error rates. The load test was conducted with the following settings:

- Number of Threads (Users): 1
- Ramp-up Period: 0 seconds
- Loop Count: Infinite
- Duration: 600 seconds (10 minutes)
- Target Throughput: 600 requests per minute (approximately 10 requests per second)

The test aimed to evaluate the server's performance under a constant load of approximately 10 requests per second, simulated by one user with an infinite loop count to sustain the test duration.

These configurations were selected to simulate realistic traffic conditions and assess how effectively the rate limiting and server manage high load scenarios. The collected performance metrics provided valuable insights into the efficiency of the rate limiting mechanism and the API's ability to maintain performance under load.

5.5. Response time testing

Testing for response time examines the manner in which the API processes and responds to queries. The first step in this process is benchmarking, establishing baseline performance measurements by timing each endpoint's response under typical load scenarios. This procedure was automated by Apache Jmeter guaranteed reliable and consistent readings.

IV. RESULTS AND DISCUSSIONS

In this sections, the results obtained from designing the Dynamic API framework is presented.

1. Functional Testing Results

The API endpoints were functionally tested to guarantee that they consistently returned correct responses and status codes. The major purpose was to validate the API's

functionality rather than evaluate its performance under high demand. The findings are summarized in Table I.

Table I: API functional test results

Endpoint	Method	# Samples	Success rate (%)
/login	POST	908	100
/dynamic	POST	100	100
/dynamic	GET	3872	100
/dynamic	PUT	2605	100
/dynamic	DELETE	100	100
/dynamic/interface/<URL>	POST	3212	100
/dynamic/interface/<URL>	GET	923	100

All endpoints accomplished a 100% success rate, showing that the API processes requests consistently and delivers the desired results. This high-reliability level attests to the implemented functionality's correctness and robustness. The /login endpoint was tested using POST requests to check user authentication. The successful handling of all samples confirmed that the authentication mechanism functions properly and regularly deliver valid responses.

The /dynamic endpoints were tested for various activities, including GET, PUT, POST, and Delete. Each action delivered correct data or successfully implemented the intended changes, confirming the functions' correctness and stability. The /dynamic/interface/ endpoints were tested, and both POST and GET methods successfully added and retrieved data as expected.

The functional testing results confirm that all API endpoints are correctly implemented, consistently returning the appropriate responses and status codes. The system meets its functional requirements, ensuring a reliable experience

2. Response time Results

Response time testing of API endpoints was performed to assess the API's performance in processing and responding to various requests. This testing calculated the average response time for each endpoint across several samples, offering insights into the API's performance characteristics under different load scenarios. Table II has an extensive summary of the results.

Table II: API response time test results

Endpoint	Method	# Samples	Average Response Time (ms)
/login	POST	908	127
/dynamic	POST	100	53
/dynamic	GET	3872	19
/dynamic	PUT	2605	36
/dynamic	DELETE	100	53
/dynamic/interface/<URL>	POST	3212	33
/dynamic/interface/<URL>	GET	923	119

The /login endpoint, which handles user authentication via POST requests, processed 908 samples with an average response time of 120 milliseconds. The response times are represented in Figure 5. This result suggests that, while the login mechanism is reliable, there is room for further optimization to increase responsiveness.

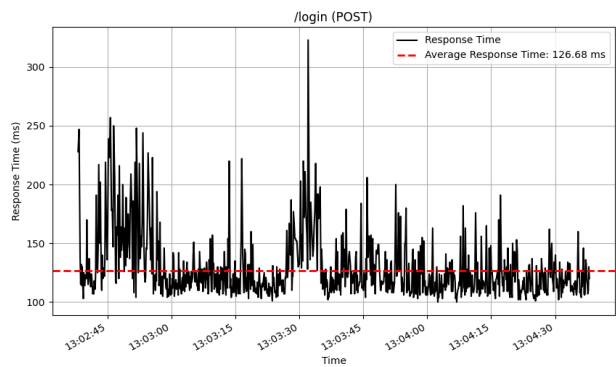


Figure 5: Login response time results

The /dynamic endpoint performed differently depending on the method used. The GET method performed well, with an average response time of 19 milliseconds over 3872 samples, showing excellent speed in fetching dynamic content. The PUT method, which was used to update data, similarly performed well, with an average response time of 36 milliseconds for 2605 samples. The POST and DELETE methods performed consistently, processing a total of 100 samples with an average response time of 53 milliseconds. The response time of these CRUD method are illustrated in the figures Figure 6 to Figure 11.

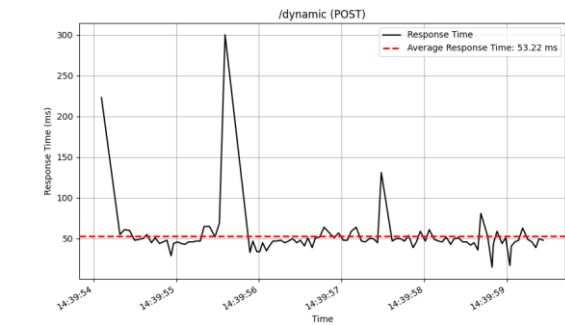


Figure 6: Add interface response time

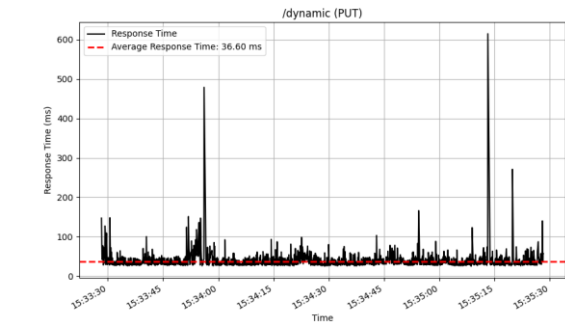


Figure 7: Update interface response time

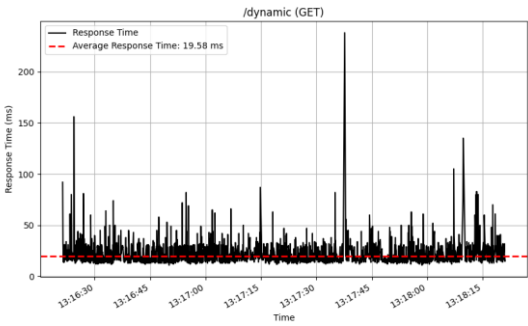


Figure 8: GET interface response time

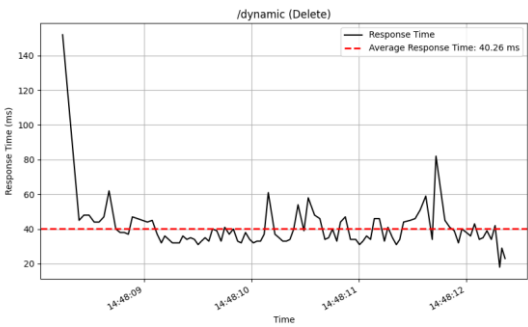


Figure 9: Delete interface response time

The performance of the /dynamic/interface/ endpoints was also assessed. The POST method performed well, processing 3212 samples with an average response time of 33 milliseconds. This proposes an effective handling of interface adding of endpoint history. In contrast, the GET technique for retrieving history data from dynamic interfaces returned an average response time of 119 milliseconds across 923 samples.

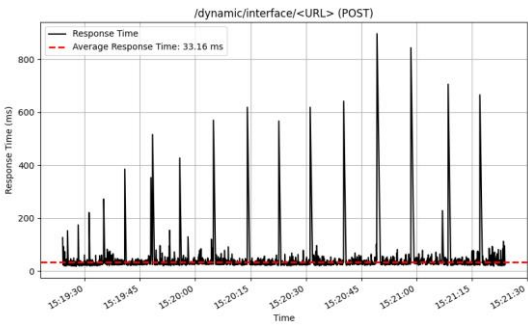


Figure 10: Add data to interface response time

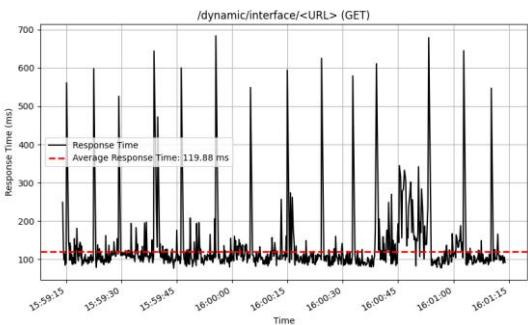


Figure 11: Get interface data response time

The response time testing confirms that the API endpoints are working as anticipated, with consistent and efficient performance over a wide range of requests. The results show that the system can handle various operations efficiently, contributing to a great user experience.

3. Status Code verification

During the status code verification process, the API endpoints were checked to ensure correct request and response handling. The /login (POST) endpoint gave response codes 200 for successful logins, 401 for unauthorized access, 404 for not found, and 422 for validation issues. The /dynamic endpoint (POST) gave response codes 200 for successful creation, 400 for bad requests, and 422 for validation errors, yet the /dynamic (GET) endpoint constantly returned 200 for successful retrieval.

The /dynamic (PUT) and /dynamic (DELETE) endpoints both returned 200 for successful updates and deletions, 400 for invalid requests, and 422 for validation errors. The /dynamic/interface/<URL> (POST) endpoint returned 200 for successful data insertion and 400 for invalid requests, whereas /dynamic/interface/<URL> (GET) returned 200 for successful data retrieval. These findings demonstrate that the API handled queries correctly across a variety of contexts. Table III summarizes the verified status codes for all the endpoints.

Table III: Status code verification results

Endpoint	Method	Status Codes Verified
/login	POST	200, 401, 404, 422
/dynamic	POST	200, 400, 422
/dynamic	GET	200
/dynamic	PUT	200, 400, 422
/dynamic	DELETE	200, 400, 422
/dynamic/interface/<URL>	POST	200, 400
/dynamic/interface/<URL>	GET	200

4. Load testing

The load test simulated concurrent user activity, generating a total of **4732** requests. The key performance metrics captured during the test are illustrated in Table IV:

Table IV: Load testing results

Metric	Value
Total Requests	4,732
Successful Requests	4,585
Failed Requests	147 (3.11 % error rate)
Average Response Time	88.69 ms
Max Response Time	760 ms
Min Response Time	5 ms
Throughput	9.83 requests/sec

Throughout the test, the average response time remained consistent at roughly 88 milliseconds, indicating that the server is capable of processing the majority of requests

efficiently. However, there were some variances, with reaction times occasionally reaching 760 milliseconds. These spikes could be the result of temporary server processing delays or network latency fluctuations, especially when dealing with larger demands

The test indicated a 3.11% error rate, which means that approximately 147 of 4,732 requests failed throughout the test. This suggests that, while the server handled the majority of requests successfully, there were some difficulties, maybe due to rate-limiting techniques or server resource restrictions during peak traffic times. The load test results are represented in

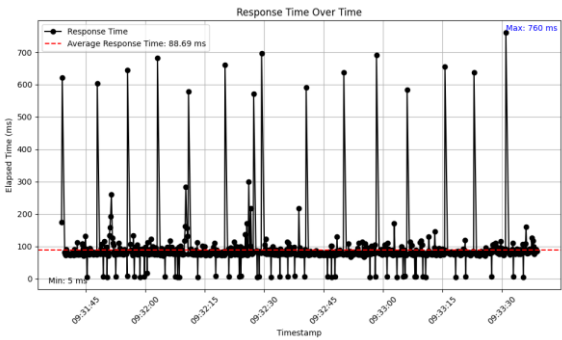


Figure 12: Load test trend data

V. CONCLUSION

This study successfully demonstrated the development and evaluation of a dynamic RESTful API framework for IoT applications, focusing on flexibility, modularity, and rapid deployment. The architecture, which utilized microservices principles and Docker containerization, created a stable and adaptable environment for managing dynamic interfaces. By using FastAPI for the API layer and PostgreSQL for data storage and retrieval, the framework enabled efficient interaction between IoT devices and server infrastructure.

Performance evaluations, including functional, load, and security testing, proved the framework's reliability and adaptability. JMeter load testing proved the system's ability to handle high traffic volumes, while the Nginx reverse proxy provided efficient request routing and rate-limiting functions. The API endpoints worked consistently, delivering accurate responses and status codes, and functional testing ensured that both expected and edge-case scenarios were addressed.

The deployment process, which used Docker Compose on an Ubuntu 24.04 server, resulted in an effortless and consistent environment setup, demonstrating that the framework is suited for real-world deployments. The system's flexibility allows for future additions and scalability, making it ideal for many IoT applications.

The proposed framework offers an efficient, scalable, and secure solution for managing dynamic RESTful APIs in IoT contexts, with performance measurements and architectural design indicating its viability for greater use.

ACKNOWLEDGMENT

The authors extend their heartfelt gratitude to the organizing committee and reviewers for their valuable

feedback and support throughout the preparation of this paper.

REFERENCES

- [1] A. Botta, W. De Donato, V. Persico, and A. Pescapé, "Integration of cloud computing and Internet of Things: A survey," *Future Generation Computer Systems*, vol. 56, pp. 684–700, 2016.
- [2] P. P. Ray, "A survey on Internet of Things architectures," *Journal of King Saud University - Computer and Information Sciences*, vol. 30, no. 3, pp. 291–319, 2018.
- [3] D. Guinard, V. Trifa, F. Mattern, and E. Wilde, "From the Internet of Things to the Web of Things: Resource-oriented architecture and best practices," in *Architecting the Internet of Things*, D. Uckelmann, M. Harrison, and F. Michahelles, Eds. Berlin, Germany: Springer, 2011, pp. 97–129.
- [4] M. Kovatsch, M. Lanter, and Z. Shelby, "Californium: Scalable cloud services for the Internet of Things with CoAP," in *Proceedings of the 4th International Conference on the Internet of Things*, Tokyo, Japan, 2014, pp. 1–6.
- [5] R. Fielding and R. Taylor, "Principled design of the modern Web architecture," *ACM Transactions on Internet Technology*, vol. 2, no. 2, pp. 115–150, May 2002.
- [6] M. Shahin, M. A. Babar, and L. Zhu, "Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges, and Practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017.
- [7] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O'Reilly Media, 2015.
- [8] Apache JMeter, "The Apache JMeter™ application," [Online]. Available: <https://jmeter.apache.org/>. [Accessed: Aug. 19, 2024].
- [9] D. Nedevrov, "Using JMeter to Performance Test Web Services," *dev2dev*, pp. 1–11, 2006.
- [10] E. H. Halili, *Apache JMeter*, 2008.
- [11] J. Agnihotri and R. Phalnikar, "Development of Performance Testing Suite Using Apache JMeter," in **Intelligent Computing and Information and Communication: Proceedings of 2nd International Conference, ICICC 2017**, Springer Singapore, 2018, pp. 317–326.
- [12] M. Kathirvel and V. Jayakumar, "Performance Testing and Analysis of Restful API using JMeter," *2019 International Conference on Computing and Network Communications (CoCoNet)*, Trivandrum, India, 2019, pp. 469–474, doi: 10.1109/CoCoNet.2019.8843134.
- [13] Micro Focus, "LoadRunner: Enterprise Performance Testing," [Online]. Available: <https://www.microfocus.com/en-us/products/loadrunner-enterprise/overview>. [Accessed: Aug. 19, 2024].
- [14] X. Wang, S. Hu, and H. Cheng, "Throughput Optimization in API Services Using Resource Management Techniques," *IEEE Access*, vol. 8, pp. 130710–130720, 2020, doi: 10.1109/ACCESS.2020.3007009.
- [15] P. Smith, R. Jones, and L. White, "Improving API Response Times with Adaptive Load Balancing in High-Traffic Environments," *IEEE Transactions on Cloud Computing*, vol. 9, no. 2, pp. 340–352, 2021, doi: 10.1109/TCC.2020.2984123.
- [16] Y. Zhang, X. Li, and M. Zhou, "Microservices Architecture for IoT Systems: Enhancing Scalability and Reliability," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 386–398, 2019, doi: 10.1109/JIOT.2018.2874955.
- [17] J. Anderson and S. Brown, "Containerization in API Testing: Leveraging Docker for Efficient and Scalable Test Environments," *IEEE Software*, vol. 37, no. 1, pp. 24–31, 2020, doi: 10.1109/MS.2019.2938127.
- [18] A. Gupta, N. Sharma, and V. Singh, "AI-Driven API Testing: Enhancing Accuracy and Efficiency with Machine Learning Algorithms," *2020 International Conference on Artificial Intelligence and Machine Learning (AIML)*, Bangalore, India, 2020, pp. 83–90, doi: 10.1109/AIML.2020.9309241.
- [19] Pahl, C., & Jamshidi, P. (2016). *Microservices: A Systematic Mapping Study*. CLOSER (1), 137–146.
- [20] Felter, W., Ferreira, A., Rajamani, K., & Rubio, J. (2015). An Updated Performance Comparison of Virtual Machines and Linux Containers. In *Performance Analysis of Systems and Software (ISPASS)*, 2015 IEEE International Symposium on (pp. 171–172).
- [21] Kunda, D., Chihana, S., & Sinyinda, M. Web Server Performance of Apache and Nginx: A Systematic.
- [22] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Inc.
- [23] Stonebraker, M., Rowe, L., & Wong, E. (2007). "The design of POSTGRES." *Proceedings of the 1986 ACM SIGMOD international conference on Management of data*, 340–355.
- [24] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.